Microsoft Graph PowerShell SDK v2 — Production Cheat Sheet

Identity Governance, Automation Pipelines, OData Optimization, Batching & Least Privilege

1. Modular SDK Installation & Setup

```
# Install only targeted sub-modules to prevent memory overhead:
Install-Module Microsoft.Graph.Authentication -Scope CurrentUser
Install-Module Microsoft.Graph.Users, Microsoft.Graph.Groups
Install-Module Microsoft.Graph.Identity.SignIns -Force
```

API Profile Selector (v1.0 vs Beta):

```
Select-MgProfile -Name 'v1.0'   # Default for production stability
Select-MgProfile -Name 'beta'   # Required for preview attributes
```

2. Production Authentication Patterns

A. Delegated (Interactive Admin with Scopes):

```
Connect-MgGraph -Scopes 'User.Read.All','Group.ReadWrite.All' -NoWelcome
```

B. Unattended Service Principal (Certificate-Based Zero Secret):

```
$authParams = @{
    ClientId              = '11111111-2222-3333-4444-555555555555'
    TenantId              = 'tenant.onmicrosoft.com'
    CertificateThumbprint = 'A1B2C3D4E5F67890...'
    NoWelcome             = $true
}
Connect-MgGraph @authParams
```

C. Azure Managed Identity (VM / Azure Function / Automation):

```
Connect-MgGraph -Identity -NoWelcome
```

D. Context Inspection & Safe Session Sign-Out:

```
$ctx = Get-MgContext
Write-Host "Connected as: $($ctx.ClientId) in Tenant: $($ctx.TenantId)"
Disconnect-MgGraph  # Clears cached token context from memory
```

3. High-Performance Querying & OData Filters

- Always filter server-side to minimize API payload size and network latency.
- Restrict returned attributes explicitly using the -Property parameter.

```
# 1. Active Users in Specific Department:
Get-MgUser -Filter "accountEnabled eq true and department eq 'IT'" `
           -Property Id,DisplayName,UserPrincipalName,Mail `
           -Top 100

# 2. Advanced Eventual Consistency ($count, startsWith, endsWith):
$count = 0
Get-MgUser -Filter "endsWith(mail, '@cloudknowledge.in')" `
           -ConsistencyLevel eventual -CountVariable count `
           -Property Id,DisplayName -All
Write-Host "Total matched records: $count"

# 3. Automated Paging for 50,000+ Objects:
# -PageSize sets HTTP batch window; -All auto-dereferences @odata.nextLink
$allUsers = Get-MgUser -All -PageSize 999 `
                       -Property Id,DisplayName,AccountEnabled
```

Standard OData Filter Operators Reference:

`eq` / `ne`	Equality / Inequality	`accountEnabled eq true`
`gt` / `ge`	Greater Than / Greater Equal	`createdDateTime gt 2026-01-01`
`lt` / `le`	Less Than / Less Equal	`signInActivity/lastSignInDateTime lt ...`
`startsWith`	Prefix String Match	`startsWith(displayName, 'SVC_')`
`endsWith`	Suffix Match (Req. ConsistencyLevel)	`endsWith(mail, '@domain.com')`
`not ( ... )`	Logical Negation	`not (userType eq 'Guest')`

4. Rate Limiting & HTTP 429 Resiliency

Microsoft Graph throttles requests per tenant and per application.
When HTTP 429 occurs, parse the Retry-After header and execute jitter backoff:

```
try {
    $result = Invoke-MgGraphRequest -Method GET -Uri $uri -ErrorAction Stop
} catch {
    if ($_.Exception.Response.StatusCode -eq 429) {
        $retrySec = $_.Exception.Response.Headers['Retry-After']
        $wait = if ($retrySec) { [int]$retrySec } else { 10 }
        Write-Warning "Throttled. Backing off for $wait seconds..."
        Start-Sleep -Seconds ($wait + (Get-Random -Minimum 1 -Maximum 3))
        # Retry request...
    }
}
```

Best Practices to Prevent Throttling:

1. Use $batch to bundle 20 calls into 1 HTTP connection.
2. Distribute high-volume workloads across multiple App Registrations.
3. Avoid parallel multi-threading against the same user/mailbox resource.
4. SDK v2 includes default 3-attempt exponential retry handler.

5. Essential Identity Cmdlet Quick Reference

Users & Lifecycle

```
Get-MgUser -UserId <UPN|Id>
// Query user object and directory attributes
New-MgUser -BodyParameter $userObj
// Provision new cloud identity
Update-MgUser -UserId <Id> -AccountEnabled:$false
// Instantly disable employee login
Set-MgUserLicense -UserId <Id> -AddLicenses ...
// Assign or revoke Microsoft 365 license SKUs
```

Groups & Memberships

```
Get-MgGroup -Filter "displayName eq 'SOC'"
// Retrieve Entra security or dynamic groups
New-MgGroupMember -GroupId <Id> -DirectoryObjectId <UId>
// Add user, device, or service principal to group
Get-MgGroupTransitiveMember -GroupId <Id>
// Resolve nested and inherited group memberships recursively
```

Applications & Service Principals

```
Get-MgApplication -Filter "appId eq '$id'"
// Retrieve multi-tenant or single-tenant App Registration
Add-MgApplicationPasswordCredential
// Generate new client secret with custom expiration date
Get-MgServicePrincipalAppRoleAssignment
// Audit API permissions and consent grants assigned to apps
```

Privileged Identity (PIM) & Security

```
Get-MgIdentityConditionalAccessPolicy
// Export all tenant Conditional Access policies for backup
Revoke-MgUserSignInSession -UserId <Id>
// Invalidate PRT tokens and kill active sessions in real-time
Open-MgPrivilegedRoleAssignmentRequest
// Schedule or request elevation for PIM eligible directory roles
```

6. High-Throughput Batching ($batch Engine)

Combine up to 20 individual REST calls into a single HTTP POST request.
Eliminates TLS handshakes and accelerates bulk automation by 15x-20x:

```
$batchPayload = @{
    requests = @(
        @{ id = '1'; method = 'GET'; url = '/users/shivam@cloudknowledge.in' }
        @{ id = '2'; method = 'GET'; url = '/groups?$filter=securityEnabled eq true' }
        @{ id = '3'; method = 'GET'; url = '/identity/conditionalAccess/policies' }
    )
} | ConvertTo-Json -Depth 5

# Execute via Invoke-MgGraphRequest engine:
$response = Invoke-MgGraphRequest -Method POST `
                                  -Uri 'https://graph.microsoft.com/v1.0/$batch' `
                                  -Body $batchPayload

# Parse batch sub-responses:
foreach ($sub in $response.responses) {
    Write-Host "Request $($sub.id) returned HTTP status $($sub.status)"
}
```

Batching Rules & Constraints:

- Maximum 20 requests per batch envelope. Sub-requests execute in parallel.
- Use 'dependsOn' array property if order of execution is mandatory.
- Sub-responses include individual HTTP status codes (200, 404, 429).

7. Zero Trust Least-Privilege Scoping Matrix

Never grant Directory.ReadWrite.All. Restrict API scopes to task domain:

User Lifecycle Audit // Read user properties, identities, and profiles	`User.Read.All`
Account Disablement // Update accountEnabled state on employee records	`User.ReadWrite.All`
Group Governance // Manage security group membership and properties	`Group.ReadWrite.All`
Credential Monitoring // Read app certificates and secret expiration dates	`Application.Read.All`
App Secret Rotation // Add or roll credentials on enterprise applications	`Application.ReadWrite.All`
Conditional Access DR // Export JSON disaster recovery policy backups	`Policy.Read.All`
CA Policy Enforcement // Deploy or modify Conditional Access policies	`Policy.ReadWrite.ConditionalAccess`
Emergency Access PIM // Activate scheduled PIM roles & assignments	`RoleManagement.ReadWrite.Directory`
Sign-in & Audit Logs // Query interactive and non-interactive telemetry	`AuditLog.Read.All`
Session Revocation // Invalidate PRT tokens and active web sessions	`User.RevokeSessions.All`
Device Identity Audit // Audit registered, joined, and hybrid devices	`Device.Read.All`
Privileged Identity Review // Audit pending and eligible PIM role assignments	`RoleEligibilitySchedule.Read.Directory`

CLOUD KNOWLEDGE ECOSYSTEM

Interactive Playbooks: https://cloudknowledge.in

YouTube Channel: @Cloud-Knowledge | GitHub: Shivi7566/graph-api-toolkit

OFFICIAL PRINTING & DESK CHEAT SHEET PARTNER

Order Laminated Desk Posters: https://printkrlo.in

PrintKrlo.in — Smart Cyber Cafe & Online Xerox Kiosk | Fast Doorstep Delivery